

Bismillah General Store

Complete Code Documentation

Full-Stack E-Commerce Grocery Store

React + Vite + Express + MongoDB

Prepared for Beginner Developers

Date: July 2026

Table of Contents

1. Project Overview & Tech Stack
2. Project Structure
3. Configuration Files
4. Frontend - Entry Point & Routing
5. Frontend - Context (State Management)
6. Frontend - API Configuration
7. Frontend - Components (Header, Footer, Chat, Misc)
8. Frontend - Pages (Auth, Frontend, Dashboard)
9. Frontend - Utility Functions
10. Frontend - Styles (SCSS)
11. Backend - Server Entry & Config
12. Backend - Database Models
13. Backend - Middleware
14. Backend - API Routes
15. How to Run the Project

1. Project Overview & Tech Stack

What is this project?

Bismillah General Store is a full-stack online grocery store where customers can browse products, add items to cart, place orders, and chat with support. Admins can manage products, orders, users, categories, stock, and view analytics through a dashboard.

Technology Stack

Layer	Technology	Purpose
Frontend	React 19 + Vite 8	UI library + fast build tool
UI Library	Ant Design (antd) 6	Pre-built UI components
Routing	React Router DOM 7	Page navigation
Styling	SCSS + Bootstrap 5	CSS framework + custom styles
Charts	Chart.js + react-chartjs-2	Dashboard analytics charts
Animations	Lottie React	Animated illustrations
Backend	Express.js 4	Node.js web framework
Database	MongoDB + Mongoose 8	NoSQL database + ODM
Auth	JWT (jsonwebtoken)	JSON Web Token authentication
File Upload	Multer + Cloudinary	Image upload & storage
Real-time	Socket.IO	Live chat feature
Password	bcryptjs	Password hashing

2. Project Structure

```

D:\Muhammad Umar\
|-- index.html                # HTML entry point
|-- package.json              # Frontend dependencies
|-- vite.config.js            # Vite build configuration
|-- jsconfig.json             # Path aliases (@/ = src/)
|-- eslint.config.js          # Code linting rules
|-- .env.sample               # Environment variables template
|-- .gitignore                # Git ignore rules
|-- firebase.json             # Firebase/Firestore config
|-- firestore.indexes.json    # Firestore indexes
|
|-- public/
|   |-- logo.png              # Favicon/logo
|
|-- src/                      # FRONTEND SOURCE CODE
|   |-- main.jsx              # React app entry point
|   |-- App.jsx               # Root component
|   |-- App.scss              # Root styles
|   |
|   |-- config/
|   |   |-- api.js            # API helper (fetch wrapper)
|   |   |-- firebase.jsx      # Firebase placeholder (removed)
|   |   |-- supabase.jsx      # Supabase placeholder (removed)
|   |   |-- global.jsx        # Global window functions
|   |
|   |-- context/
|   |   |-- AppProvider.jsx    # Wraps Auth + Cart contexts
|   |   |-- Auth.jsx          # Authentication state
|   |   |-- Cart.jsx           # Shopping cart state
|   |
|   |-- components/
|   |   |-- Header/
|   |   |   |-- index.jsx      # Header wrapper
|   |   |   |-- Navbar.jsx     # Navigation bar
|   |   |-- Footer/
|   |   |   |-- index.jsx      # Footer wrapper
|   |   |   |-- Copyright.jsx  # Copyright footer
|   |   |-- Chat/
|   |   |   |-- ChatWidget.jsx # Customer chat widget
|   |   |-- Misc/
|   |   |   |-- ProtectedRoute.jsx # Auth guard
|   |   |   |-- ScreenLoader.jsx # Loading screen
|   |   |   |-- NoPage.jsx     # 404 page
|   |
|   |-- pages/
|   |   |-- Routes.jsx         # Main route definitions
|   |   |-- Auth/              # Auth pages (Login, Register, ForgotPassword)
|   |   |-- Frontend/          # Public pages (Home, Shop, Cart, Checkout, etc.)
|   |   |-- Dashboard/        # Admin dashboard pages
|   |
|   |-- utils/
|   |   |-- format.js          # Date/currency formatting
|   |   |-- chat.js            # Chat API helpers
|   |   |-- recycleBin.js      # Recycle bin helpers
|   |
|   |-- scss/                  # SCSS style files
|   |-- assets/                # Static assets (JSON animations)
|
|-- server/                    # BACKEND SOURCE CODE

```

```
| |-- index.js           # Express server entry
| |-- package.json      # Server dependencies
| |-- .env              # Server environment variables
| |-- config/
| | |-- db.js           # MongoDB connection
| | |-- cloudinary.js    # Cloudinary config
| |-- middleware/
| | |-- auth.js         # JWT auth + role middleware
| | |-- upload.js       # Multer file upload config
| |-- models/           # Mongoose schemas
| | |-- User.js, Product.js, Order.js, Category.js, Chat.js, StockMovement.js
| |-- routes/           # API routes
| | |-- auth.js, products.js, orders.js, categories.js,
| | |-- users.js, stock.js, dashboard.js, chat.js, upload.js
|
|-- Data/               # Documentation output folder
```

3. Configuration Files

package.json (Frontend)

```
{
  "name": "todos",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",          // Start dev server
    "build": "vite build",  // Build for production
    "lint": "eslint .",    // Run linter
    "preview": "vite preview" // Preview production build
  },
  "dependencies": {
    "antd": "^6.3.7",      // UI component library
    "bootstrap": "^5.3.8", // CSS framework
    "chart.js": "^4.5.1",  // Charting library
    "lottie-react": "^2.4.1", // Animations
    "react": "^19.2.6",    // React core
    "react-chartjs-2": "^5.3.1", // React Chart.js wrapper
    "react-dom": "^19.2.6", // React DOM renderer
    "react-router-dom": "^7.15.0", // Client-side routing
    "sass": "^1.99.0"      // SCSS compiler
  }
}
```

vite.config.js

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import path from 'path';

export default defineConfig({
  plugins: [react()],
  resolve: {
    alias: {
      '@': path.resolve(__dirname, './src'), // @/ maps to src/
    },
  },
})
```

jsconfig.json

```
{
  "compilerOptions": {
    "paths": {
      "@/*": ["./src/*"] // Enables @/ imports in IDE
    }
  }
}
```

.env.sample

```
VITE_FIREBASE_API_KEY=          # Firebase config (legacy)
VITE_FIREBASE_AUTH_DOMAIN=
VITE_SUPABASE_URL=              # Supabase config (legacy)
VITE_SUPABASE_PUBLISHABLE_KEY=
```

firebase.json

```
{
  "firestore": { "indexes": "firestore.indexes.json" },
  "projects": { "default": "grocery-store-94de0" }
}
```

index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <link rel="icon" type="image/png" href="/logo.png" />
  <meta name="theme-color" content="#1d3557" />
  <title>Bismillah General Store</title>
  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&display=swap"
rel="stylesheet" />
</head>
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
</html>
```

4. Frontend - Entry Point & Routing

src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.jsx'
import './config/global.jsx'
import "bootstrap/dist/js/bootstrap.bundle.js"
import { BrowserRouter } from 'react-router-dom'
import AppProvider from './context/AppProvider.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <AppProvider>
        <App />
      </AppProvider>
    </BrowserRouter>
  </StrictMode>,
)
```

Explanation: This is the entry point. It wraps the entire app in:

- `StrictMode` - React's development checks
- `BrowserRouter` - Enables client-side routing
- `AppProvider` - Provides Auth + Cart context to all components

src/App.jsx

```
import './App.scss'
import Routes from './pages/Routes'
import { ConfigProvider } from 'antd'
import { useAuth } from './context/Auth'
import ScreenLoader from '@components/Misc/ScreenLoader'

const App = () => {
  const { isAppLoading } = useAuth()
  return (
    <ConfigProvider theme={{ components: { Button: { controlOutlineWidth: 0 } } }}>
      {!isAppLoading ? <Routes /> : <ScreenLoader />}
    </ConfigProvider>
  )
}
export default App
```

Explanation: Shows a loading screen while checking auth status, then renders Routes. Uses Ant Design's ConfigProvider for theme customization.

src/pages/Routes.jsx


```

import { Navigate, Route, Routes } from 'react-router-dom'
import Auth from './Auth'
import Dashboard from './Dashboard'
import Frontend from './Frontend'
import NoPage from '@components/Misc/NoPage'
import ProtectedRoute from '@components/Misc/ProtectedRoute'
import { useAuth } from '@context/Auth'

const Index = () => {
  const { isAuth } = useAuth()
  return (
    <Routes>
      <Route path="/" element={<Frontend />} />          {/* Public pages */}
      <Route path="auth/*" element={!isAuth ? <Auth /> : <Navigate to="/" />} /> /* Auth pages */}
      <Route path="dashboard/*" element={<ProtectedRoute adminOnly element={<Dashboard />} />} /> /* Admin
only */}
      <Route path="*" element={<NoPage />} />          {/* 404 page */}
    </Routes>
  )
}
export default Index

```

Route Structure:

URL Pattern	Component	Access
/	Frontend (Home)	Public
/shop	Shop page	Public
/shop/:id	Product Details	Public
/cart	Shopping Cart	Public
/checkout	Checkout	Logged-in users
/orders	My Orders	Logged-in users
/about	About page	Public
/contact	Contact page	Public
/auth/login	Login	Guests only
/auth/register	Register	Guests only
/dashboard	Admin Dashboard	Admin only

5. Frontend - Context (State Management)

src/context/AppProvider.jsx

```
import AuthContext from './Auth'
import CartContext from './Cart'

const AppProvider = ({ children }) => {
  return (
    <AuthContext>
      <CartContext>
        {children}
      </CartContext>
    </AuthContext>
  )
}
export default AppProvider
```

Explanation: Wraps children with both Auth and Cart context providers. This is a common pattern called "Context Composition."

src/context/Auth.jsx

```
import { createContext, useContext, useEffect, useState } from 'react'
import { api } from '@config/api'

const Auth = createContext()

const AuthContext = ({ children }) => {
  const [state, setState] = useState({
    isAuthenticated: false, user: null, userRole: null, isAppLoading: true
  })

  useEffect(() => {
    // Check if user was previously logged in (from localStorage)
    const token = localStorage.getItem('token')
    const savedUser = localStorage.getItem('user')
    if (token && savedUser) {
      const user = JSON.parse(savedUser)
      setState({ isAuthenticated: true, user, userRole: user.role, isAppLoading: false })
    } else {
      setState({ isAuthenticated: false, user: null, userRole: null, isAppLoading: false })
    }
  }, [])

  const handleLogin = async (email, password) => {
    const res = await api.post('/auth/login', { email, password })
    localStorage.setItem('token', res.token)
    localStorage.setItem('user', JSON.stringify(res.user))
    setState({ isAuthenticated: true, user: res.user, userRole: res.user.role, isAppLoading: false })
    return res
  }

  const handleRegister = async (fullName, email, password, phone, avatarFile) => {
    const formData = new FormData()
    formData.append('fullName', fullName)
    formData.append('email', email)
    formData.append('password', password)
    if (phone) formData.append('phone', phone)
    if (avatarFile) formData.append('avatar', avatarFile)

    const res = await fetch('http://localhost:5000/api/auth/register', {
      method: 'POST',
      body: formData,
    }).then(r => r.json())

    localStorage.setItem('token', res.token)
    localStorage.setItem('user', JSON.stringify(res.user))
    setState({ isAuthenticated: true, user: res.user, userRole: res.user.role, isAppLoading: false })
    return res
  }

  const handleLogout = () => {
    localStorage.removeItem('token')
    localStorage.removeItem('user')
    setState({ isAuthenticated: false, user: null, userRole: null, isAppLoading: false })
  }

  return (
    <Auth.Provider value={{ ...state, handleLogin, handleRegister, handleLogout }}>
      {children}
    </Auth.Provider>
  )
}
```

```
    )  
  }  
  
  export default AuthContext  
  export const useAuth = () => useContext(Auth)
```

How Auth Works:

1. On app load, check `localStorage` for saved token and user
2. On login, send credentials to `POST /api/auth/login`
3. Server returns JWT token + user data
4. Token is stored in `localStorage` and attached to all API requests
5. On logout, clear `localStorage` and reset state

`src/context/Cart.jsx`

```

import { createContext, useContext, useEffect, useState } from 'react'

const Cart = createContext()

const CartContext = ({ children }) => {
  const [cart, setCart] = useState(() => {
    const saved = localStorage.getItem('cart')
    return saved ? JSON.parse(saved) : []
  })

  useEffect(() => {
    localStorage.setItem('cart', JSON.stringify(cart))
  }, [cart])

  const addToCart = (product, quantity = 1) => {
    setCart(prev => {
      const existing = prev.find(item => item._id === product._id)
      if (existing) {
        return prev.map(item =>
          item._id === product._id
            ? { ...item, quantity: item.quantity + quantity }
            : item
        )
      }
      return [...prev, { ...product, quantity }]
    })
  }

  const removeFromCart = (productId) => {
    setCart(prev => prev.filter(item => item._id !== productId))
  }

  const updateQuantity = (productId, quantity) => {
    if (quantity <= 0) { removeFromCart(productId); return }
    setCart(prev => prev.map(item =>
      item._id === productId ? { ...item, quantity } : item
    ))
  }

  const clearCart = () => setCart([])

  const cartTotal = cart.reduce((sum, item) => sum + item.price * item.quantity, 0)
  const cartCount = cart.reduce((sum, item) => sum + item.quantity, 0)

  return (
    <Cart.Provider value={{ cart, addToCart, removeFromCart, updateQuantity, clearCart, cartTotal, cartCount }}>
      {children}
    </Cart.Provider>
  )
}

export default CartContext
export const useCart = () => useContext(Cart)

```

How Cart Works:

- Cart data is persisted in `localStorage`
- `addToCart` - Adds product or increases quantity if already in cart

- `removeFromCart` - Removes item by product ID
- `updateQuantity` - Sets exact quantity (removes if 0)
- `cartTotal` & `cartCount` - Computed values

6. Frontend - API Configuration

src/config/api.js

```
const API_BASE = 'http://localhost:5000/api'

const getToken = () => localStorage.getItem('token')

const headers = (extra = {}) => {
  const h = { ...extra }
  const token = getToken()
  if (token) h['Authorization'] = `Bearer ${token}`
  return h
}

export const api = {
  get: async (path) => {
    const res = await fetch(`${API_BASE}${path}`, { headers: headers() })
    if (!res.ok) {
      const err = await res.json().catch(() => ({ error: 'Request failed' }))
      throw new Error(err.error || 'Request failed')
    }
    return res.json()
  },

  post: async (path, body, isFormData = false) => {
    const opts = { method: 'POST', headers: headers() }
    if (isFormData) {
      opts.body = body
    } else {
      opts.headers['Content-Type'] = 'application/json'
      opts.body = JSON.stringify(body)
    }
    const res = await fetch(`${API_BASE}${path}`, opts)
    if (!res.ok) {
      const err = await res.json().catch(() => ({ error: 'Request failed' }))
      throw new Error(err.error || 'Request failed')
    }
    return res.json()
  },

  put: async (path, body, isFormData = false) => { /* similar to post */ },
  delete: async (path) => { /* similar to get with DELETE method */ },
}
```

Explanation: This is a custom fetch wrapper that:

- Automatically adds the JWT token to all requests
- Supports JSON and FormData (for file uploads)
- Handles errors uniformly
- All API calls go to `http://localhost:5000/api`

src/config/global.jsx

```
import { message } from 'antd'

// Generates random unique ID
window.getRandomId = () => {
  return Math.random().toString(36).slice(2) + Math.random().toString(36).slice(2)
}

// Validates email format
window.isEmailVaild = (email) => {
  const re = /^[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}$/
  return re.test(String(email).toLowerCase())
}

// Shows toast notification (success, error, info, warning)
window.toastify = (msg, type) => message[type](msg)
```

Explanation: Adds global helper functions to the window object for use anywhere in the app.

src/config/firebase.jsx & src/config/supabase.jsx

```
// Firebase removed - using Express + MongoDB backend instead
export const db = null
export const auth = null
export const storage = null

// Supabase removed - using Cloudinary for image storage instead
export const supabase = null
```

Note: These are legacy placeholders kept to prevent import errors. The actual backend uses Express + MongoDB.

7. Frontend - Components

src/components/Header/Navbar.jsx

```
// Key features:
// - Responsive navbar with mobile drawer
// - Shows cart badge with item count
// - Different buttons for logged-in vs guest users
// - Admin users see "Dashboard" button, customers see "My Orders"
// - Mobile-friendly hamburger menu

const Navbar = () => {
  const { isAuthenticated, user, userRole, handleLogout } = useAuth()
  const { cartCount } = useCart()
  const [drawerOpen, setDrawerOpen] = useState(false)

  return (
    <nav className="navbar navbar-expand-lg bg-primary navbar-dark">
      <Link className="navbar-brand fw-bold" to="/">Bismillah General Store</Link>
      {/* Mobile toggle button */}
      {/* Navigation links: Home, Shop, About, Contact */}
      {/* Cart badge, Login/Register or Dashboard/Logout buttons */}
      {/* Mobile drawer menu */}
    </nav>
  )
}
```

src/components/Footer/Copyright.jsx

```
// Three-column footer:
// 1. Store name and description
// 2. Quick Links (Home, Shop, About, Contact)
// 3. Customer Service (Cart, My Orders, Support)
// Copyright notice at bottom
```

src/components/Misc/ProtectedRoute.jsx

```
import { useAuth } from '@context/Auth'
import { Navigate } from 'react-router-dom'
import ScreenLoader from './ScreenLoader'

const ProtectedRoute = ({ element, adminOnly }) => {
  const { isAuthenticated, userRole, isAppLoading } = useAuth()
  if (isAppLoading) return <ScreenLoader /> // Show loading
  if (!isAuthenticated) return <Navigate to="/auth/login" /> // Redirect to login
  if (adminOnly && userRole !== 'admin') return <Navigate to="/" /> // Non-admin
  return element
}
export default ProtectedRoute
```

Explanation: Route guard that checks if user is authenticated. If `adminOnly` prop is true, also checks if user has admin role.

src/components/Chat/ChatWidget.jsx

```
// Floating chat widget for customers:  
// - Fixed button in bottom-right corner  
// - Opens modal with message list  
// - Sends messages via API  
// - Polls for new messages every 5 seconds  
// - Shows unread message count  
// - Only visible to logged-in users
```

8. Frontend - Pages

8.1 Auth Pages

src/pages/Auth/Login/index.jsx

```
// Login page with email/password form
// - Validates inputs
// - Calls handleLogin from Auth context
// - Shows toast notifications
// - Redirects to home on success
// - Link to register page
```

src/pages/Auth/Register/index.jsx

```
// Registration page with:
// - Profile photo upload (circular preview)
// - Full name, email, password, confirm password
// - Validation: name 3+ chars, valid email, password 6+ chars
// - Sends FormData to backend (supports file upload)
// - First registered user becomes admin automatically
```

src/pages/Auth/ForgotPassword/index.jsx

```
// Password reset page
// - Simple email input
// - Currently directs user to contact admin
```

8.2 Frontend (Public) Pages

src/pages/Frontend/Home/index.jsx

```
// Home page with 3 sections:
// 1. Hero section - Welcome banner with "Start Shopping" button
// 2. Categories - Grid of category cards (fetched from API)
// 3. Featured Products - Up to 8 featured products in a grid
// Clicking a category navigates to /shop?category=NAME
// Clicking a product navigates to /shop/:id
```

src/pages/Frontend/Shop/index.jsx

```
// Shop page with:  
// - Product grid (fetches all products from /api/products)  
// - Search bar (filters by product name)  
// - Sort dropdown (Newest, Price Low-High, Price High-Low)  
// - Category filter from URL params  
// - "Out of Stock" overlay on unavailable products  
// - Click product to view details
```

src/pages/Frontend/ProductDetails/index.jsx

```
// Product detail page:  
// - Large product image  
// - Name, category, price, description  
// - Stock status (In Stock / Out of Stock)  
// - Quantity selector (step 0.1 for KG items)  
// - "Add to Cart" button  
// - "Continue Shopping" button
```

src/pages/Frontend/Cart/index.jsx

```
// Shopping cart page:  
// - List of cart items with image, name, price, quantity controls  
// - Remove item button  
// - Order summary sidebar (subtotal, shipping note, total)  
// - "Proceed to Checkout" button  
// - "Continue Shopping" button  
// - Empty cart state with "Start Shopping" button
```

src/pages/Frontend/Checkout/index.jsx

```
// Checkout page with two columns:  
// LEFT: Shipping information form  
//   - Full name, phone, address, city, ZIP code  
// RIGHT: Invoice/receipt preview  
//   - Bill number, date, customer info  
//   - Item table with quantities and prices  
//   - Grand total  
//   - "Place Order" button  
//  
// On order placement:  
// 1. Validates all required fields  
// 2. POST /api/orders with cart items + shipping info  
// 3. Clears cart  
// 4. Redirects to orders page
```

src/pages/Frontend/Orders/index.jsx

```
// Customer orders page:  
// - Lists all orders as receipt-style cards  
// - Each card shows: Bill No, Status, Date, Items table, Total  
// - Status colors: pending=orange, confirmed=blue, shipped=purple, delivered=green, cancelled=red  
// - "Cancel Order" button (with confirmation) - restores stock  
// - Redirects to login if not authenticated
```

8.3 Dashboard (Admin) Pages

src/pages/Dashboard/index.jsx

```
// Admin dashboard layout with sidebar:  
// - Sidebar menu with icons  
// - Mobile responsive (drawer menu)  
// - Routes: Home, Products, Recycle Bin, Stock, Orders, Sales, Bills, Chat, Categories, Users  
// - "View Store" link  
// - Logout button
```

src/pages/Dashboard/Home/index.jsx

```
// Dashboard home with analytics:  
// STAT CARDS: Products, Orders, Pending, Delivered, Revenue, Profit, Stock Profit, Low Stock, Users,  
Categories  
// CHARTS (using Chart.js):  
// 1. Daily Revenue (Line chart)  
// 2. Daily Orders (Line chart)  
// 3. Daily Profit (Line chart)  
// 4. Monthly Revenue (Line chart)  
// 5. Monthly Orders (Line chart)  
// 6. Monthly Profit (Line chart)  
// 7. Top 5 Products by Revenue (Bar chart)  
// 8. Product Demand - Quantity Sold (Bar chart)  
// 9. Revenue Share by Top Products (Doughnut chart)  
// 10. All Products Selling Rate (Horizontal Bar chart)
```

src/pages/Dashboard/Products/index.jsx

```
// Product management:  
// - Table with: Image, Name, Category, Sale Price, Cost Price, Profit, Stock, Featured, Actions  
// - "Add Product" button opens modal form  
// - Edit product (same form, pre-filled)  
// - Delete moves to Recycle Bin (soft delete)  
// - Image upload to Cloudinary  
// - Form fields: Name, Category, Sale Price, Cost Price, Unit, Stock, Featured toggle, Description, Image
```

src/pages/Dashboard/Orders/index.jsx

```
// Order management:  
// - Table with: Order ID, Customer, Items, Total, Status (dropdown), Date, Actions  
// - Status can be changed via dropdown (pending/confirmed/shipped/delivered/cancelled)  
// - "View Details" opens bill modal  
// - Print bill (opens new window with formatted bill)  
// - Delete order (restores stock)  
// - Shows profit per item
```

src/pages/Dashboard/Categories/index.jsx

```
// Category management:  
// - Table with: Image, Name, Slug, Created, Actions  
// - Add/Edit category via modal  
// - Image upload to Cloudinary  
// - Delete moves to Recycle Bin
```

src/pages/Dashboard/Users/index.jsx

```
// User management:  
// - Table with: Avatar, Name, Email, Phone, Role, Status, Password (hidden), Joined, Actions  
// - Edit user via modal (name, email, role, status, password, avatar)  
// - Block/Unblock user toggle  
// - Delete user (cannot delete admins)  
// - Cannot modify own account
```

src/pages/Dashboard/Stock/index.jsx

```
// Stock management:  
// - Summary cards: In Stock, Low Stock ( $\leq 5$ ), Out of Stock  
// - Product table with stock status (color-coded)  
// - "Adjust Stock" modal: select product, add/remove, quantity, note  
// - Stock Movement History table  
// - Toggle to show/hide out-of-stock products
```

src/pages/Dashboard/Sales/index.jsx

```
// Sales records:  
// - Summary stats: Total Revenue, Profit, Daily Revenue, 7-day Revenue, Orders, Active, Cancelled, Items Sold, Avg Order Value  
// - Monthly revenue grid  
// - Date range filter  
// - Sales history table with bill view modal
```

src/pages/Dashboard/Bills/index.jsx

```
// Bill records:  
// - Table of all orders as bills  
// - Bill view modal with print functionality  
// - Shows item-level profit calculation
```

src/pages/Dashboard/Chat/index.jsx

```
// Chat support (admin side):  
// - Left sidebar: list of customer conversations  
// - Right panel: message thread  
// - WhatsApp-style UI  
// - Send messages to customers  
// - Close chat conversations  
// - Auto-refresh every 3-5 seconds  
// - Responsive mobile view
```

src/pages/Dashboard/RecycleBin/index.jsx

```
// Recycle bin (unified for all entity types):  
// - Tabs: Products, Categories, Orders, Users  
// - Each tab shows deleted items with Restore/Delete permanently  
// - Bulk operations: Restore All, Delete All  
// - Select multiple items for bulk actions  
// - "Empty Recycle Bin" to delete everything  
// - "Show All" toggle to see all items (including active)
```

9. Frontend - Utility Functions

src/utils/format.js

```
// formatDate - "29-Jul-2026 02:30:45 PM"
export const formatDate = (ts) => {
  if (!ts) return '-'
  const d = new Date(ts)
  const date = d.toLocaleDateString('en-GB', { day: '2-digit', month: 'short', year: 'numeric' }).replace(/
/g, '-')
  const time = d.toLocaleTimeString('en-GB', { hour: '2-digit', minute: '2-digit', second: '2-digit', hour12:
true })
  return `${date} ${time}`
}

// formatCurrency - "1,234.56"
export const formatCurrency = (value) => {
  if (value == null || isNaN(value)) return '0.00'
  return Number(value).toLocaleString('en-US', { minimumFractionDigits: 2, maximumFractionDigits: 2 })
}
```

src/utils/chat.js

```
// Chat helper functions using the API wrapper:
// createOrGetChat(userId, userEmail, userName) - Creates or retrieves active chat
// sendMessage(chatId, senderId, senderName, senderRole, text) - Send a message
// subscribeToMessages(chatId, callback) - Polls messages every 5 seconds
// subscribeToUserChats(userId, callback) - Polls chats every 5 seconds
// markMessagesAsRead(chatId, userId) - Marks messages as read
// closeChat(chatId) - Closes a chat (admin only)
```

src/utils/recycleBin.js

```
// Recycle bin helper functions:
// moveToRecycleBin(collectionName, itemId) - Soft delete
// restoreFromRecycleBin(collectionName, itemId) - Restore
// permanentlyDelete(collectionName, itemId) - Hard delete
// deleteFromCloudinary(imageUrl) - Delete image from Cloudinary
// batchRestore(items) - Restore multiple items
// batchPermanentDelete(items) - Delete multiple items
```

10. Frontend - Styles (SCSS)

src/App.scss

```
@import "../scss/bootstrap"; // Bootstrap customizations
@import "../scss/auth"; // Auth page styles
@import "../scss/dashboard"; // Dashboard styles
@import "../scss/screen-loader"; // Loading screen styles
@import "../scss/store"; // Store frontend styles

* { margin: 0; padding: 0; box-sizing: border-box; }
body { font-family: 'Inter', sans-serif; }
#root { display: flex; flex-direction: column; min-height: 100vh; }
main { flex: 1 0 auto; }
footer { flex-shrink: 0; margin-top: auto; }
a { text-decoration: none; }
```

SCSS Files:

File	Contains
_bootstrap.scss	Bootstrap utility overrides
_auth.scss	Login/Register page styles
_dashboard.scss	Dashboard layout, charts, chat styles
_screen-loader.scss	Full-screen loading spinner
_store.scss	Store pages (hero, products, cart, footer)

11. Backend - Server Entry & Config

server/package.json

```
{
  "name": "bismillah-store-server",
  "type": "module",
  "scripts": {
    "start": "node index.js",
    "dev": "node --watch index.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",          // Password hashing
    "cloudinary": "^2.5.1",       // Image storage
    "cors": "^2.8.5",             // Cross-origin requests
    "dotenv": "^16.4.7",         // Environment variables
    "express": "^4.21.2",         // Web framework
    "jsonwebtoken": "^9.0.2",    // JWT auth
    "mongodb": "^7.5.0",         // MongoDB driver
    "mongoose": "^8.9.5",        // MongoDB ODM
    "multer": "^1.4.5-lts.1",    // File upload
    "socket.io": "^4.8.1"        // Real-time communication
  }
}
```

server/index.js

```

import 'dotenv/config'
import express from 'express'
import cors from 'cors'
import { createServer } from 'http'
import { Server } from 'socket.io'
import connectDB from './config/db.js'

const app = express()
const server = createServer(app)
const io = new Server(server, {
  cors: { origin: process.env.FRONTEND_URL || '*', methods: ['GET', 'POST'] },
})

app.use(cors())
app.use(express.json({ limit: '10mb' }))
app.use(express.urlencoded({ extended: true }))

// API Routes
app.use('/api/auth', authRoutes)
app.use('/api/products', productRoutes)
app.use('/api/categories', categoryRoutes)
app.use('/api/orders', orderRoutes)
app.use('/api/users', userRoutes)
app.use('/api/dashboard', dashboardRoutes)
app.use('/api/stock', stockRoutes)
app.use('/api/chats', chatRoutes)
app.use('/api/upload', uploadRoutes)

app.get('/api/health', (req, res) => res.json({ status: 'ok' }))

// Socket.IO for real-time chat
io.on('connection', (socket) => {
  socket.on('join-chat', (chatId) => socket.join(chatId))
  socket.on('send-message', (data) => io.to(data.chatId).emit('new-message', data))
  socket.on('mark-read', (chatId) => io.to(chatId).emit('messages-read', chatId))
})

connectDB().then(() => {
  server.listen(PORT, () => console.log(`Server running on port ${PORT}`))
})

```

server/config/db.js

```

import mongoose from 'mongoose'

const connectDB = async () => {
  const conn = await mongoose.connect(process.env.MONGODB_URI, {
    serverSelectionTimeoutMS: 30000,
    socketTimeoutMS: 45000,
  })
  console.log(`MongoDB connected: ${conn.connection.host}`)
}
export default connectDB

```

server/config/cloudinary.js

```
import { v2 as cloudinary } from 'cloudinary'

cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
  api_key: process.env.CLOUDINARY_API_KEY,
  api_secret: process.env.CLOUDINARY_API_SECRET,
})
export default cloudinary
```

Server Environment Variables (.env):

MONGODB_URI - MongoDB connection string
JWT_SECRET - Secret key for JWT tokens
JWT_EXPIRES_IN - Token expiry (default: 7d)
CLOUDINARY_CLOUD_NAME - Cloudinary cloud name
CLOUDINARY_API_KEY - Cloudinary API key
CLOUDINARY_API_SECRET - Cloudinary API secret
PORT - Server port (default: 5000)
FRONTEND_URL - Frontend URL for CORS

12. Backend - Database Models

server/models/User.js

```
const userSchema = new mongoose.Schema({
  uid:      { type: String, required: true, unique: true }, // Custom unique ID
  fullName: { type: String, required: true },
  email:    { type: String, required: true, unique: true },
  password: { type: String, required: true },              // bcrypt hashed
  phone:    { type: String, default: '' },
  avatar:   { type: String, default: '' },                 // Cloudinary URL
  role:     { type: String, enum: ['admin', 'customer'], default: 'customer' },
  status:   { type: String, enum: ['active', 'blocked'], default: 'active' },
  isDeleted: { type: Boolean, default: false },           // Soft delete
  deletedAt: { type: Number, default: null },
  deletedBy: { type: String, default: '' },
}, { timestamps: true })
```

server/models/Product.js

```
const productSchema = new mongoose.Schema({
  name:      { type: String, required: true },
  description: { type: String, default: '' },
  price:     { type: Number, required: true },           // Sale price
  costPrice: { type: Number, default: 0 },               // Purchase cost
  stock:     { type: Number, default: 0 },
  unit:      { type: String, enum: ['KG', 'pcs', 'L', 'dozen', 'pack'], default: 'pcs' },
  category:  { type: String, default: '' },
  images:    [{ type: String }],                        // Cloudinary URLs
  featured:  { type: Boolean, default: false },
  featuredOrder: { type: Number, default: 0 },
  position:  { type: Number, default: 0 },
  isDeleted:  { type: Boolean, default: false },
  deletedAt:  { type: Number, default: null },
  deletedBy:  { type: String, default: '' },
}, { timestamps: true })
```

server/models/Order.js

```
const orderSchema = new mongoose.Schema({
  userId: { type: String, required: true },
  userEmail: { type: String, default: '' },
  items: [{ type: mongoose.Schema.Types.Mixed }], // Array of product items
  total: { type: Number, required: true },
  status: { type: String, enum: ['pending', 'confirmed', 'shipped', 'delivered', 'cancelled'], default: 'pending' },
  shipping: {
    fullName: { type: String, default: '' },
    phone: { type: String, default: '' },
    address: { type: String, default: '' },
    city: { type: String, default: '' },
    zip: { type: String, default: '' },
  },
  isDeleted: { type: Boolean, default: false },
  deletedAt: { type: Number, default: null },
  deletedBy: { type: String, default: '' },
}, { timestamps: true })
```

server/models/Category.js

```
const categorySchema = new mongoose.Schema({
  name: { type: String, required: true },
  slug: { type: String, required: true, unique: true }, // URL-friendly name
  icon: { type: String, default: '' }, // Emoji icon
  image: { type: String, default: '' }, // Cloudinary URL
  isDeleted: { type: Boolean, default: false },
  deletedAt: { type: Number, default: null },
  deletedBy: { type: String, default: '' },
}, { timestamps: true })
```

server/models/Chat.js

```
// Chat schema - represents a conversation
const chatSchema = new mongoose.Schema({
  userId: { type: String, required: true },
  userEmail: { type: String, default: '' },
  userName: { type: String, default: '' },
  status: { type: String, enum: ['active', 'closed'], default: 'active' },
  lastMessage: { type: mongoose.Schema.Types.Mixed, default: null },
  unreadCount: { type: Number, default: 0 },
}, { timestamps: true })

// Message schema - individual messages in a chat
const messageSchema = new mongoose.Schema({
  chatId: { type: ObjectId, ref: 'Chat', required: true },
  senderId: { type: String, required: true },
  senderName: { type: String, default: '' },
  senderRole: { type: String, enum: ['admin', 'customer'], default: 'customer' },
  text: { type: String, required: true },
  read: { type: Boolean, default: false },
}, { timestamps: true })
```

server/models/StockMovement.js

```
const stockMovementSchema = new mongoose.Schema({
  productId: { type: ObjectId, ref: 'Product', required: true },
  productName: { type: String, default: '' },
  type: { type: String, enum: ['in', 'out', 'adjustment'], default: 'adjustment' },
  quantity: { type: Number, required: true },
  previousStock: { type: Number, default: 0 },
  newStock: { type: Number, default: 0 },
  note: { type: String, default: '' },
}, { timestamps: true })
```

13. Backend - Middleware

server/middleware/auth.js

```
import jwt from 'jsonwebtoken'
import User from '../models/User.js'

// Middleware: Verify JWT token and attach user to request
export const protect = async (req, res, next) => {
  const header = req.headers.authorization
  if (!header || !header.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Not authorized, no token' })
  }
  const token = header.split(' ')[1]
  const decoded = jwt.verify(token, process.env.JWT_SECRET)
  const user = await User.findOne({ uid: decoded.uid, isDeleted: false }).select('-password')
  if (!user) return res.status(401).json({ error: 'User not found' })
  if (user.status === 'blocked') return res.status(403).json({ error: 'Account blocked' })
  req.user = user
  next()
}

// Middleware: Check if user is admin
export const adminOnly = (req, res, next) => {
  if (req.user?.role !== 'admin') {
    return res.status(403).json({ error: 'Admin access required' })
  }
  next()
}

// Helper: Create JWT token
export const signToken = (uid, role) => {
  return jwt.sign({ uid, role }, process.env.JWT_SECRET, { expiresIn: '7d' })
}
```

server/middleware/upload.js

```
import multer from 'multer'

const storage = multer.memoryStorage() // Store files in memory
const upload = multer({
  storage,
  limits: { fileSize: 10 * 1024 * 1024 } // 10MB max
})
export default upload
```

Explanation: Multer stores uploaded files in memory (not disk). The buffer is then converted to base64 and uploaded to Cloudinary.

14. Backend - API Routes

Authentication Routes

Method	Endpoint	Auth	Description
POST	/api/auth/register	No	Register new user (FormData with optional avatar)
POST	/api/auth/login	No	Login with email/password
GET	/api/auth/me	Yes	Get current user
PUT	/api/auth/change-password	Yes	Change password

Product Routes

Method	Endpoint	Auth	Description
GET	/api/products	Yes	List products (optional: ?featured=true, ?all=true)
GET	/api/products/:id	Yes	Get product by ID
POST	/api/products	Admin	Create product (FormData with image)
PUT	/api/products/:id	Admin	Update product
DELETE	/api/products/:id	Admin	Soft delete (move to recycle bin)
PUT	/api/products/:id/restore	Admin	Restore from recycle bin
DELETE	/api/products/:id/permanent	Admin	Permanent delete + Cloudinary cleanup
PUT	/api/products/bulk/restore	Admin	Restore all deleted products
DELETE	/api/products/bulk/permanent	Admin	Permanent delete all deleted products

Order Routes

Method	Endpoint	Auth	Description
GET	/api/orders	Yes	List orders (customers see own, admin sees all)
POST	/api/orders	Yes	Place order (deducts stock)
PUT	/api/orders/:id	Yes	Update status (customers can only cancel)
DELETE	/api/orders/:id	Admin	Soft delete + restore stock

Category, User, Stock, Dashboard, Chat, Upload Routes

Route Group	Key Endpoints
Categories	GET/POST/PUT/DELETE /api/categories (+ restore, bulk operations)
Users	GET/PUT/DELETE /api/users (+ block, restore, bulk operations)
Stock	GET /api/stock (products + movements), PUT /api/stock/:id (adjust)
Dashboard	GET /api/dashboard/stats, GET /api/dashboard/chart-data
Chat	GET/POST /api/chats, messages, read, close
Upload	POST /api/upload (image), DELETE /api/upload/:publicId

15. How to Run the Project

Prerequisites

- Node.js 18+ installed
- MongoDB running locally or MongoDB Atlas account
- Cloudinary account (for image uploads)

Backend Setup

1. Open terminal and navigate to server/ folder:
`cd server`
2. Create .env file with these variables:
`MONGODB_URI=mongodb+srv://your-connection-string`
`JWT_SECRET=your-secret-key`
`JWT_EXPIRES_IN=7d`
`CLOUDINARY_CLOUD_NAME=your-cloud-name`
`CLOUDINARY_API_KEY=your-api-key`
`CLOUDINARY_API_SECRET=your-api-secret`
`PORT=5000`
`FRONTEND_URL=http://localhost:5173`
3. Install dependencies:
`npm install`
4. Start the server:
`npm run dev`
(Server runs on `http://localhost:5000`)

Frontend Setup

1. Open a new terminal and navigate to project root:
`cd "D:\Muhammad Umar"`
2. Install dependencies:
`npm install`
3. Start the dev server:
`npm run dev`
(Frontend runs on `http://localhost:5173`)
4. Open browser and go to:
`http://localhost:5173`

First Time Setup

- 1. Register the first account - it automatically becomes ADMIN
- 2. Login with admin credentials
- 3. Go to Dashboard to:
 - Add categories (e.g., Groceries, Beverages, Snacks)
 - Add products with images, prices, and stock
- 4. Test as customer:
 - Browse shop, add items to cart
 - Place an order
 - Use chat support

Available Scripts

Command	Location	Description
npm run dev	Root	Start Vite dev server
npm run build	Root	Build for production
npm run lint	Root	Run ESLint
npm run dev	server/	Start Express server with auto-reload
npm start	server/	Start production server

Bismillah General Store - Code Documentation
Generated for Beginner Developers
All source files included with explanations